

PRÁCTICAS DE SISTEMAS OPERATIVOS II

PRÁCTICA LINUX DE GRUPO

Aparcando

1. Enunciado.

En esta práctica vamos a simular, mediante procesos de UNIX, la gestión de memoria de un sistema operativo que use particiones de tamaño dinámico. Para ello, se establece el símil con el problema de elegir el mejor aparcamiento para un coche de todos los posibles en una acera. Según se va ejecutando el programa, se ha de ver una imagen parecida a la siguiente:

The screenshot displays the 'parking' simulation program with a blue background and white text. It shows four stages of parking adjustments, each with a header bar and a corresponding XML-like output line. The header bars are orange with black text, and the output lines are blue with white text. The stages are: 1. 'PRIMER AJUSTE' (First Adjustment) with 'Atendido: 151' and 'En cola: 323'. 2. 'SIGUIENTE AJUSTE' (Next Adjustment) with 'Atendido: 129' and 'En cola: 343'. 3. 'MEJOR AJUSTE' (Better Adjustment) with 'Atendido: 163' and 'En cola: 309'. 4. 'PEOR AJUSTE' (Worst Adjustment) with 'Atendido: 145' and 'En cola: 328'. The output lines contain XML-like tags with car IDs and positions. At the bottom right, there is a button labeled 'PARKING VER. 2.2'.

```
Atendido: 151 PRIMER AJUSTE En cola: 323
<\153|>.....154.....
<|#####154#####/>

Atendido: 129 SIGUIENTE AJUSTE En cola: 343
<\#####130#####|><\#####131#####|><|###132###/><-#133##\><|###134#####/>

Atendido: 163 MEJOR AJUSTE En cola: 309
<\#####164#####|><\###165###|><|##166###/><|167##/><-#####168###\>

Atendido: 145 PEOR AJUSTE En cola: 328
<\#####142#####|><\#####143#####|><\##144##|><-149\> <|146#/><|147#/></148->

PARKING VER. 2.2
```

El programa constará de un único fichero fuente, "**parking.c**", cuya adecuada compilación producirá el ejecutable "**parking**". Respetad las mayúsculas/minúsculas de los nombres, si las hubiere.

Para simplificar la realización de la práctica, se os proporciona una biblioteca estática de funciones "**libparking.a**" que debéis enlazar con vuestro módulo objeto para generar el ejecutable. Gracias a ella, algunas de las funciones necesarias para realizar la práctica no las tendréis que programar, sino que os bastará incluir la biblioteca cuando compiléis el programa. La línea de compilación del programa podría ser:

```
gcc parking.c libparking.a -o parking -lm
```

Disponéis, además, de un fichero de cabecera, "**parking.h**", donde se encuentran definidas, entre otras cosas, las macros que usa la biblioteca, definición de tipos de datos y prototipos de funciones.

En la imagen puede observarse de color marrón lo que simulan ser cuatro aceras. El algoritmo que se usa para aparcar los coches en cada una corresponde, de arriba a abajo, al *primer ajuste*, *siguiente ajuste*, *mejor ajuste* y *peor ajuste*, tal y como probablemente habréis visto en la teoría de Sistemas Operativos I. Cada acera consta de ochenta caracteres (de $x=0$ a $x=79$), numerados de izquierda a derecha. La coordenada y puede tomar valores entre 0 y 2, siendo 0 la calzada justo al lado de la acera, 1, la posición sobre las rayas de aparcamiento y 2, el carril de circulación. Los coches tienen longitud entera, medida en caracteres.

Quizá de los cuatro algoritmos el que mayor dificultad presente sea el del *siguiente ajuste*. Cuando un coche llega a aparcar, de todos los huecos en los que cabe el coche, se elige el primero que aparece a partir de la posición en la que aparcó el último coche. Si se llega al final de la acera, se vuelve a empezar por el principio. El algoritmo no presenta mayor problema salvo cuando el último coche que aparcó ya se ha ido y también está libre la posición actual. En ese caso, se empieza a buscar por el comienzo del hueco donde se halla la posición actual.

En el caso de los algoritmos del *mejor ajuste* y del *peor ajuste*, si hay empate entre dos o más huecos, se elegirá entre ellos siguiendo el criterio del primer ajuste.

Una vez elegido un hueco con espacio suficiente para aparcar el coche, se aparcará ajustando el coche a la izquierda del hueco en cualquiera de los algoritmos.

Si cuando llega un coche, no existe un hueco de tamaño suficiente para que aparque, el coche se pondrá en una cola a la espera de que otro coche salga del aparcamiento. Esta cola seguirá una disciplina FIFO.

Los coches tienen un número que los identifica. **Los coches deben aparcar en orden consecutivo.** Vuestro programa debe garantizar que el orden se respeta, pues de no hacer nada, algunos coches podrían adelantarse debido al reparto de CPU. También tienen los coches un color y una velocidad propias.

Para que en la forma de desarrollarse de cada algoritmo no influya la suerte, en cada ejecución concreta llegarán los mismos coches, con idéntico tiempo de servicio e intervalo de llegada entre ellos a cada una de las cuatro aceras. Para el tiempo de servicio y el intervalo de llegada entre coches, se usará una distribución exponencial. Para la longitud de los coches, la distribución será uniforme, con una longitud comprendida 7 y 17 caracteres.

Para facilitar la tarea, tenéis a vuestra disposición una biblioteca de funciones de enlazado estático "**libparking.a**" y un fichero de cabeceras, "**parking.h**". Gracias a la biblioteca, muchas de las funciones no las tendréis que programar, sino invocarlas a la biblioteca. También estará a cargo de la biblioteca muchos de los detalles de la práctica: llegadas y salidas de coches, gestión de la cola de espera, etc. Una descripción detallada de las funciones de la biblioteca aparece más abajo.

Además del proceso principal-padre, necesitaréis crear algunos procesos clon-hijos. En concreto, existirán unos procesos **chóferes** que se dedicarán a aparcar y desapparcar los coches. Estos procesos siguen una estrategia denominada *cuadrilla de trabajadores* (*pool of workers*). Consiste en que se quedan en espera a que les llegue una petición de aparcar o desapparcar un coche. Una vez realizada la tarea, vuelven a quedarse a la espera de recibir otra solicitud. También existe un clon-hijo adicional del proceso padre que se encargará de contar el tiempo que dura la simulación e invocar a la

función **PARKING_fin**. Es posible que necesitéis, adicionalmente, un proceso gestor para tramitar los mensajes recibidos de la biblioteca a través del buzón.

El programa, desde vuestro punto de vista, se simplifica bastante. El proceso padre ha de:

1. Comprobar los argumentos pasados por la línea de órdenes.
2. Registrar las manejadoras.
3. Crear los recursos **IPC** necesarios: array de semáforos, buzón y zona de memoria compartida.
4. Llamar a la función **PARKING_inicio**.
5. Crear los procesos chóferes y el proceso que avisa.
6. Invocar la función **PARKING_simulaciOn**.

Para que el programa funcione correctamente, es necesario crear un array de semáforos, un buzón de paso de mensajes y una zona de memoria compartida privados. Parte de estos recursos los necesita la biblioteca y el resto que reservéis los podéis usar para sincronizar y comunicar los procesos entre sí. En concreto, hay dos funciones auxiliares que os indican cuántos semáforos necesita la biblioteca (**PARKING_getNSemAforos**) y cuántos bytes de memoria compartida requiere la biblioteca (**PARKING_getTamaNoMemoriaCompartida**). Tanto los semáforos como la zona de memoria usados por la biblioteca estarán situados al inicio del recurso reservado.

Por su parte, la vida de los procesos chóferes transcurre plácida y tranquila, como ya se ha explicado. Duermen hasta recibir una petición de aparcar o desaparcar e invocan las funciones **PARKING_aparcar** y **PARKING_desaparcar**, respectivamente. Solamente deben tener cuidado de no llamar a la función de aparcamiento hasta que los coches con un número inferior al suyo ya la hayan invocado y recibido confirmación de la biblioteca.

Cuando hay que aparcar o desaparcar un coche, la biblioteca manda un mensaje al buzón de tipo **PARKING_MSG** y subtipos **PARKING_MSGSUB_APARCAR** y **PARKING_MSGSUB_DESAPARCAR**, respectivamente. El formato de estos mensajes se puede deducir de la definición de **struct PARKING_mensajeBiblioteca** que se encuentra en el fichero "**parking.h**".

De haber varias peticiones sin atender, dependiendo de cómo se haya invocado la práctica, se debe seguir la política siguiente:

1. De tipo FIFO si no se especifica nada en la línea de órdenes.
2. De prioridad para los aparcamientos si especificamos el argumento "**PA**".
3. De prioridad para desaparcar si se especifica el argumento "**PD**".

Para lograrlo, se puede usar un proceso gestor que tramite los mensajes, los reescriba, renumere y reenvíe o, cualquier otro método que logre los resultados que se pretenden.

Cada coche que participa en la simulación tiene un manejador opaco de tipo **HCoche** para poder trabajar con él. Existen funciones que permiten conocer las características de los coches mediante su manejador. Un listado exhaustivo se ofrece más abajo.

Se deben programar cuatro funciones, una por algoritmo, que serán llamadas por la biblioteca cada vez que un coche llegue. Dichas *funciones de rellamada* son registradas en la función **PARKING_inicio** de la biblioteca.

El prototipo de las funciones de rellamada se describe aquí:

- **int funciOn_llegada(HCoche hc);**

La biblioteca llamará a una de estas funciones cuando llegue un nuevo coche. La respuesta que debe devolver vuestro código indicará qué se debe hacer con ese coche. -1 indica que no hay sitio para él y debe encolarse. Un valor entre 0 y 79 significa la posición a partir de la cual se debe aparcar el coche. Además, si devolvemos -2, indicaremos a la biblioteca que no queremos procesar más coches de ese algoritmo, lo que puede ser útil para anular uno o varios algoritmos para depurar los demás.

Las funciones **PARKING_aparcar** y **PARKING_desaparcar** son especiales puesto que gestionan el movimiento de los coches automáticamente mediante funciones de rellamada a vuestro código. Estos son sus prototipos con la correspondiente explicación:

- **int PARKING_aparcar(HCoche, void *datos, TIPO_FUNCION_APARCAR_COMMIT, TIPO_FUNCION_PERMISO_AVANCE, TIPO_FUNCION_PERMISO_AVANCE_COMMIT);**

La biblioteca puede llamar a la función de llegada en cualquier orden y es responsabilidad del programador que los coches se aparquen en orden numérico. Para ello, se debe invocar la función **PARKING_aparcar** del coche que toque y no invocar a la función del coche siguiente hasta haber recibido la rellamada de *aparcar commit*. Esta rellamada indica que la biblioteca ha tomado nota de la anterior llamada y ya se puede invocar al siguiente.

El avance del coche también se regula mediante dos funciones de rellamada que se especifican en esta función. Cada vez que un coche quiere avanzar, la biblioteca invoca a nuestra función de *permiso avance*. Debemos bloquearnos, sin consumir CPU, hasta que el avance sea seguro. En esos momentos, la biblioteca produce el avance en el coche y nos avisa a través de la función *permiso avance commit*.

- **int PARKING_desaparcar(HCoche, void *datos, TIPO_FUNCION_PERMISO_AVANCE, TIPO_FUNCION_PERMISO_AVANCE_COMMIT);**

La descripción de esta función es equivalente a la de la función anterior, con la salvedad de que, al no importar el orden en que se desaparca, no es necesaria una función de confirmación (commit) en este caso.

Características adicionales que programar

- El programa no debe consumir CPU apreciablemente en los modos de retardo mayor o igual que 1. Para comprobar el consumo de CPU, podéis usar la orden **top**.
- **IMPORTANTE:** Aunque no se indique explícitamente en el enunciado, parece obvio que se necesitarán objetos de sincronización en diferentes partes del programa.
- Se permite hacer semiespera ocupada cuando un **chófer** espera a que le toque a su coche aparcar, pero de hacerse así, la nota puede verse rebajada.
- El tiempo de ejecución de la práctica será de 30 segundos. El proceso encargado de llevar la cuenta no puede usar la llamada **sleep**, sino hacerlo mediante la señal **SIGALRM**.

El programa debe estar preparado para que, si el usuario pulsa las teclas **CTRL-C** desde el terminal, la ejecución del programa termine en ese momento y adecuadamente. Ni en una terminación como esta, ni en una normal, deben quedar procesos en ejecución ni mecanismos **IPC** sin haber sido borrados del sistema. Este es un aspecto muy importante y se penalizará bastante si la práctica no lo cumple.

Es probable que necesitéis semáforos para sincronizar adecuadamente la práctica. Se declarará un array de semáforos de tamaño adecuado a vuestros requerimientos, los primeros de los cuales se reservarán para el funcionamiento interno de la biblioteca, como se indicó más arriba. El resto, podéis usarlos libremente. Del mismo modo, la memoria compartida reservada se divide en dos partes, la primera de las cuales es para uso interno de la biblioteca.

Argumentos:

Parking acepta un mínimo de dos y un máximo de cuatro argumentos por la línea de órdenes. Si no se introducen argumentos, se imprimirá un mensaje con la forma de uso del programa por el canal de error estándar. En el caso de teclear dos argumentos, el primer argumento será un número entero mayor o igual que cero relacionado con la rapidez (o retardo) con que se producen los acontecimientos en el programa. Cero indica la máxima rapidez y números mayores suponen una rapidez progresivamente menor. El segundo argumento es el número de chóferes que se usarán en esta ejecución. Finalmente, si son más de dos los argumentos introducidos, los dos primeros son idénticos al caso anterior y el resto podrá ser una combinación cualquiera de:

- **D**, indicando que se desea que el programa produzca información de depuración por el canal de errores estándar.
- **PA**, para que se dé prioridad a los que aparcen.
- **PD**, para que se dé prioridad a los que desaparcen.

La única combinación que no se permitirá será **PA** y **PD** a la vez, por contradictoria. Por ejemplo, para una velocidad 10, cinco chóferes, con información de depuración y prioridad para los que desaparcen, se podría invocar así:

parking 10 5 PD D

Funciones de la biblioteca:

Las funciones proporcionadas por la biblioteca son las que a continuación aparecen. De no indicarse nada, las funciones devuelven -1 en caso de error:

- **int PARKING_inicio(int ret, TIPO_FUNCION_LLEGADA f_llegadas[], int semAforos, int buzOn, int zona, int debug)**
El proceso padre debe llamar a esta función cuando desee que la simulación comience. La función devuelve -1 si se ha producido un error. Los argumentos son:
 - **ret**: lentitud de ejecución. Es el valor que se ha pasado como primer argumento al programa.
 - **f_llegadas**: es un array de cuatro elementos. Contiene los punteros a las funciones de vuestro programa que queréis que la biblioteca invoque cada vez que llega un coche nuevo para cada algoritmo. El prototipo de las funciones se ha especificado más arriba.
 - **semAforos**: identificador del array de semáforos devuelto por **semget**.
 - **buzOn**: idem para el buzón de paso de mensajes.
 - **zona**: idem para la zona de memoria compartida (no es el puntero, sino lo que devuelve **shmget**).
 - **debug**: flag que indica si se desea que la biblioteca produzca información de depuración por el canal de errores estándar.

- **void PARKING_aparcar(HCoche hc, void *datos
, TIPO_FUNCION_APARCAR_COMMIT fCommit
, TIPO_FUNCION_PERMISO_AVANCE fPermisoAvance
, TIPO_FUNCION_PERMISO_AVANCE_COMMIT fPermisoAvanceCommit)**

Se invoca esta función cuando queremos que aparezca por la pantalla la animación de un coche aparcando y siempre respetando el orden numérico de los coches:

- **hc:** manejador del coche que queremos aparcar.
- **datos:** puntero de datos que podemos asociar al coche para recuperarlos en cualquier otro momento.
- **fCommit:** manejadora que será invocada cuando la biblioteca haya anotado la salida del coche.
- **fPermisoAvance:** manejadora que es llamada por la biblioteca antes de efectuar un movimiento del coche para aparcar. A través del manejador del coche que se nos pasa, podremos obtener su información. En particular, su posición (funciones **PARKING_getX()** y **PARKING_getY()**), la posición a la que desea ir (funciones **PARKING_getX2()** y **PARKING_getY2()**), el puntero a los datos que hemos asociado al coche (función **PARKING_getDatos()**), etc. La función se debe bloquear sin consumo de CPU hasta que sea seguro para el coche el avance.
- **fPermisoAvanceCommit:** esta manejadora será invocada por la biblioteca cuando ha hecho efectivo el avance del coche. En este caso, las coordenadas **(x,y)** se corresponden con la posición ya avanzada del coche y **(x2,y2)** son las coordenadas de la posición antes de avanzar.

- **void PARKING_desaparcar(HCoche hc, void *datos
, TIPO_FUNCION_PERMISO_AVANCE fPermisoAvance
, TIPO_FUNCION_PERMISO_AVANCE_COMMIT fPermisoAvanceCommit)**

Idéntica a la función anterior, sólo que para que se muestre la salida de un coche. En este caso no aparece la función *commit* pues se hace innecesaria, al no ser un requisito que desaparquen en un determinado orden.

- **int PARKING_getNumero(HCoche hc)**
- **int PARKING_getLongitud(HCoche hc)**
- **int PARKING_getPosiciOnEnAcera(HCoche hc)**
- **int PARKING_getColor(HCoche hc)**
- **int PARKING_getX(HCoche hc)**
- **int PARKING_getY(HCoche hc)**
- **int PARKING_getX2(HCoche hc)**
- **int PARKING_getY2(HCoche hc)**
- **int PARKING_getAlgoritmo(HCoche hc)**
- **unsigned long PARKING_getTServ(HCoche hc)**
- **void *PARKING_getDatos(HCoche hc)**

Todas estas funciones dan información relativa al coche cuyo manejador se pasa como parámetro. Dicha información es, respectivamente:

- número del coche.
- longitud del coche.
- posición **x** en la acera donde ese coche va a aparcar o -1 si todavía no está decidido.
- color del coche.
- coordenada **x** de la posición del coche donde se encuentra.
- lo mismo para la coordenada **y**.
- coordenada **x** de la posición a la que quiere avanzar el coche (función de permiso de avance) o de la que viene (función de confirmación de permiso de avance).
- idem coordenada **y**.

- algoritmo al que pertenece el coche (**PRIMER_AJUSTE**, **SIGUIENTE_AJUSTE**, **MEJOR_AJUSTE** y **PEOR_AJUSTE**), según vienen definidas en el fichero de cabecera.
 - tiempo de servicio en unidades de tiempo de la simulación.
 - puntero de datos que se le ha asignado al coche en la función de aparcar.
- **int PARKING_isAceraOcupada(int algoritmo,int pos)**
Devuelve verdadero si la posición (**pos**: 0-79) de la acera (**algoritmo**: 0-3) indicadas en sus argumentos está ocupada por un coche, desde el punto de vista de la biblioteca. Solamente debe usarse como depuración. El estado de las aceras en cada momento lo debe gestionar vuestra aplicación con sus propias variables.
- **int PARKING_getTamaNoMemoriaCompartida()**
- **int PARKING_getNSemAforos()**
Estas funciones devuelven los requerimientos de la biblioteca respecto a la memoria compartida (bytes) o número de semáforos.
- **int PARKING_simulaciOn()**
El proceso padre debe llamar a esta función después de haber iniciado todo lo necesario para que la simulación se lleve a cabo. Permanecerá dentro de la función hasta que la simulación acabe. No obstante, la simulación no acabará en una ejecución normal hasta que todos los coches pendientes de desaparecer se hayan ido.
- **int PARKING_fin(int tipo)**
Se debe llamar a esta función cuando se desee terminar la simulación. Si se trata de una terminación normal, se le pasará un 1. Si es por un error, se le pasará un cero.
- **void pon_error(char *mensaje)**
Pone un mensaje de error en la parte inferior de la pantalla y espera a que el usuario pulse "Intro". La podéis usar para depurar.

Estad atentos pues pueden ir saliendo versiones nuevas de la biblioteca para corregir errores o dotarla de nuevas funciones.

Notas acerca de las funciones:

Respecto a la sincronización interna de la biblioteca, se usa un semáforo reservado para conseguir atomicidad en la actualización de la pantalla (**semPantalla**, iniciado a 1), otro semáforo para evitar consumo de CPU innecesario en la simulación (**semSim**, iniciado a 0) y cuatro semáforos adicionales para secciones críticas para cada uno de los cuatro algoritmos (**semAlgoritmo[]**, iniciados a 1). Para que las sincronizaciones que de seguro deberéis hacer en vuestro código estén en sintonía con las de la biblioteca, os ofrecemos ahora un pseudocódigo de algunas de las funciones que realiza la biblioteca y están reguladas por los semáforos.

- * **PARKING_inicio:**
 - da valores iniciales a los recursos IPC de la biblioteca
 - dar valor inicial a los parámetros de la simulación
 - iniciar el generador de números aleatorios
 - limpiar la pantalla, poner mensaje inicial y dibujar
- * **PARKING_fin:**

- si es normal, marcar acabando
- si es por error, hacer que el cursor sea visible

* PARKING_simulaciOn:

- comienzo del bucle de simulación
- cálculo del próximo suceso (llegada o salida)
- dormir hasta que llegue el suceso o se avise mediante semSim
- avanzar el reloj de la simulación
- para todos los algoritmos a
 - W(semAlgoritmo[a])
 - pasar los recién aparcados a la lista de aparcados
 - destruir los coches recién desaparcados
 - S(semAlgoritmo[a])
- si toca acabar, salir del bucle
- para todos los algoritmos a
 - mandar mensaje por cada coche cuyo tiempo de espera ha expirado (desaparcar)
- si estamos acabando, continuar el bucle
- mientras el tiempo de llegada del próximo coche sea menor que el reloj actual, crear un nuevo coche y encolarlo en la lista de espera
- para todos los coches que estén en la lista de espera
 - W(semAlgoritmo[a])
 - llamar a la función de llegada del algoritmo para ese coche
 - si la posición devuelta es -2
 - poner t_siguiente llegada muy alto; S(semAlgoritmo[a]); break
 - si la posición calculada es incorrecta
 - poner el error; S(semAlgoritmo[a]); break
 - si la posición calculada es -1
 - S(semAlgoritmo[a]); break
 - dibujar la reserva
 - mandar mensaje de aparcar
 - S(semAlgoritmo[a])
- fin del bucle de la simulación

* PARKING_aparcar:

- si no le toca a este coche, poner el error
- W(semAlgoritmo[a])
- incrementar el número que toca
- S(semAlgoritmo[a])
- almacenar el puntero de los datos de usuario pasado en el coche
- desde la pos=79 hasta la posición en que el coche tiene que aparcar
 - pausa de avance
 - llamar a permiso de avance
 - dibujar el coche
 - llamar a permiso de avance commit
 - si pos=79, llamar a aparcar commit
- hacer lo mismo para los dos avances verticales para acabar de aparcar
- W(semAlgoritmo[a])
- encolar el coche en los recién aparcados
- avisar con semSim
- S(semAlgoritmo[a])

* PARKING_desaparcar:

- permiso avance, dibujo
 - y llamar a avance commit para el primer avance vertical
- permiso avance para el segundo avance vertical
- W(semAlgoritmo[a])
- dibujar el coche
- liberar la reserva de la acera
- llamar a permiso avance commit
- S(semAlgoritmo[a])

- hasta que el coche desaparece: pausa, permiso avance, dibujo, llamar a `avance commit`
- `W(semAlgoritmo[a])`
- meter el coche en la lista de recién desaparcados
- avisar con `semSim`
- `S(semAlgoritmo[a])`

En esta práctica no se podrán usar ficheros para nada, salvo que se indique expresamente. Las comunicaciones de PIDs o similares entre procesos, si hicieran falta, se harán mediante *mecanismos IPC*.

Siempre que en el enunciado o LPEs se diga que se puede usar **`sleep()`**, se refiere a la *llamada al sistema*, no a la orden de la línea de órdenes.

Los mecanismos IPC (semáforos, memoria compartida y paso de mensajes) son recursos muy limitados. Es por ello, que vuestra práctica solo podrá usar un conjunto de semáforos, un buzón de paso de mensajes y una zona de memoria compartida como máximo. Además, si se produce cualquier error o se finaliza normalmente, los recursos creados han de ser eliminados. Una manera fácil de lograrlo es registrar la señal **SIGINT** para que lo haga y mandársela uno mismo si se produce un error, aunque hay mejores soluciones.

Biblioteca de funciones "libparking.a"

Con esta práctica se trata de que aprendáis a sincronizar y comunicar procesos en UNIX entre sí. Su objetivo no es la programación, aunque es inevitable que tengáis que programar. Es por ello que se os suministra una biblioteca estática de funciones ya programadas para tratar de que no debáis preocuparos por la presentación por pantalla, la gestión de estructuras de datos (colas, pilas, ...), etc. También servirá para que se detecten de un modo automático errores que se produzcan en vuestro código. Para que vuestro programa funcione, necesitáis la propia biblioteca **"libparking.a"** y el fichero de cabecera **"parking.h"**. La biblioteca funciona con los códigos de VT100/xterm, por lo que debéis adecuar la emulación de vuestro terminal.

2. Pasos recomendados para la realización de la práctica.

Aunque ya deberíais ser capaces de abordar la práctica sin ayuda, aquí van unas guías generales:

1. Hacer el código necesario para gestionar los argumentos que se le pasan al programa.
2. Crear los recursos IPC necesarios (semáforos, memoria compartida, buzón de paso de mensajes), y comprobad que se crean bien, con **ipcs**. Es preferible, para que no haya interferencias, que los defináis privados.
3. Registrad **SIGINT** para que cuando se pulse **CTRL+C** se ordene la terminación, se limpien los procesos hijo y se eliminen los recursos IPC. Lograr que, si el programa acaba normalmente o se produce cualquier error, también se eliminen los recursos (mandad una señal **SIGINT** al proceso padre en esos casos).
4. Llamar a la función **PARKING_inicio** en **main**. Debe aparecer la pantalla de bienvenida y, pasados dos segundos, dibujarse la pantalla. Para el array de funciones de llegada, usad funciones que

nada más tengan un **pause()** dentro, de momento. Todos los procesos que se creen a continuación en sucesivos puntos deben crearse después de haber llamado a **PARKING_inicio**.

5. Podéis ejecutar la práctica, a partir de ahora, redirigiendo la salida de errores a un fichero para que no se os emborrone la pantalla.
6. A continuación, haced que el padre llame a **PARKING_simulaciOn()**
7. Los siguientes pasos conllevan resolver la práctica solo para un algoritmo y con un chófer, para ir, poco a poco, aumentando la dificultad. Para ello, solo programad la función de llegada del primer algoritmo. El resto de funciones de llegada, que devuelvan -2 para indicar a la biblioteca que no queréis, de momento, tratar con ellas. Y el chófer, que solo lea el mensaje y lo imprima en la pantalla.
8. Que el chófer llame ahora a **PARKING_aparcar**. Podéis hacer que las funciones de commits y permisos, de momento, solo pongan un mensaje de depuración.
9. Que el chófer distinga entre los dos tipos de mensajes y llame a **PARKING_aparcar** o **PARKING_desaparcar**, según corresponda. Ya deberíais ser capaces de ver a los coches aparcar y salir, cuando toque.
10. Las cosas se van a complicar ahora, cuando añadáis varios chóferes: puede haber errores de orden de aparcamiento y de choques. Debéis introducir mecanismos de sincronización para que todo funcione.
11. Acabad la práctica y probadla a velocidad normal y a velocidad cero.
12. Meted la opción de diferentes políticas de gestión de la cola de tareas de los chóferes: FIFO, prioridad de aparcamientos y prioridad de desaparcamientos.
13. Añadid el resto de algoritmos.
14. Diseñad la forma de acabar sin problemas y llamad a la función **PARKING_fin()**.
15. Pulid los últimos detalles.

3. Presentación y Plazo.

Esta práctica de grupo para el Sistema Operativo LINUX, se entregará a través de una tarea creada al efecto en la plataforma moodle de la asignatura (<http://studium.usal.es>), con lo que el plazo límite de presentación viene marcado por dicha tarea. Debéis consultar los plazos de la misma para conocer específicamente las fechas de entrega. No obstante, se estima que, desde su planteamiento, deberá ser entregada en **cuatro semanas** aproximadamente.

4. Normas de presentación.

1. Tenéis que entregar el código fuente y la documentación en un solo archivo (comprimido en .zip o .tgz), con el nombre indicado. Además, sólo podéis subir uno a Moodle.
2. En la cabecera del código fuente, se especificará los años del curso (2025-2026 por ejemplo), el número y tipo de práctica en grupo (primera, segunda, etc. para Linux o Windows), el título, el código de grupo de prácticas asignado, los nombres y apellidos de los integrantes del grupo, y la fecha.
3. Se incluirá junto con la práctica la documentación indicada más adelante. Dentro del código fuente se incluirán los comentarios relativos a novedades introducidas sobre lo visto en clase y sólo eso. Esto se hace para facilitar el proceso de corrección. En la mayor parte de los casos, no añadiréis nada por ser obvio, pero a veces es bueno comentar los algoritmos utilizados.

4. Si la práctica incluye salida por pantalla, hay que ajustarse exacta y literalmente a lo especificado, carácter a carácter, línea a línea, por si se automatiza parte del proceso de corrección. Es evidente que debéis borrar, comentar o dejar inactivos todos los comentarios de depuración que aparezcan por la pantalla, si es que los habéis usado.
5. Es necesario que los integrantes del grupo de prácticas informen previamente al profesor de quienes son para que se les asigne un código de grupo.
6. El no cumplimiento de los plazos y modo de entrega dará lugar a una reducción de la nota o a que la práctica esté suspensa, dependiendo de la gravedad de la falta.

Documentación Adicional:

Además de estas normas, en esta práctica se debe entregar un esquema donde aparezcan los recursos IPC usados (semáforos con sus valores iniciales, memoria compartida con sus variables, usos y significados, buzones y mensajes pasados, en caso que se usen), y un pseudo código sencillo para cada proceso con las operaciones *wait* y *signal*, *send* y *receive* realizadas sobre ellos. Por ejemplo, si se tratara de sincronizar dos procesos C y V para que produjeran alternativamente consonantes y vocales, comenzando por una consonante, deberíais entregar algo parecido a esto:

SEMÁFOROS Y VALOR INICIAL: SC=1, SV=0.

SEUDOCÓDIGO:

C	V
===	===
Por_siempre_jamás	Por _siempre_jamás
{	{
W(SC)	W(SV)
escribir_consonante	escribir_vocal
S(SV)	S(SC)
}	}

Daos cuenta que lo que importa en el pseudo código es la sincronización. El resto puede ir muy esquemático. Un buen esquema os facilitará muchísimo la defensa.

5. Evaluación de la práctica.

Dada la dificultad para la corrección de programación en paralelo, el criterio que se seguirá para la evaluación de la práctica será: si...

- a) la práctica cumple las especificaciones de este enunciado y,
- b) la práctica no falla en ninguna de las ejecuciones a las que se somete y,
- c) no se descubre en la práctica ningún fallo de construcción que pudiera hacerla fallar, por muy remota que sea esa posibilidad...

...se aplicará el principio de "presunción de inocencia" y la práctica estará aprobada. La nota, a partir de ahí, dependerá de la simplicidad de las técnicas de sincronización usadas, la corrección en el tratamiento de errores, la cantidad y calidad del trabajo realizado, etc.

6. Notas sobre el desarrollo de la práctica LPEs.

- I. ¿Se puede usar la biblioteca en un Linux de 64 bits? No. La librería usada en esta práctica está compilada para 32 bits.
- II. ¿Se puede proporcionar la biblioteca para el Sistema Operativo X, procesador Y? Por problemas de eficiencia en la gestión y mantenimiento del código no se proporcionará la biblioteca más que para Linux de 32 bits (con procesador de Intel o AMD). A veces podéis lograr encontrar una solución mediante el uso de máquinas virtuales.
- III. ¿Dónde poner un semáforo? Dondequiera que uséis la frase, "el proceso debe esperar hasta que..." es un buen candidato a que aparezca una operación **WAIT** sobre un semáforo. Tenéis que plantearos a continuación qué proceso hará **SIGNAL** sobre ese presunto semáforo y cuál será su valor inicial.
- IV. Si ejecutáis la práctica en *segundo plano* (con ampersand (&)) es normal que al pulsar CTRL+C el programa no reaccione. El terminal sólo manda SIGINT a los procesos que estén en primer plano. Para probarlo, mandad el proceso a primer plano con fg %y pulsad entonces CTRL+C.
- V. Un "truco" para que sea menos penoso el tratamiento de errores consiste en dar valor inicial a los identificadores de los recursos IPC igual a -1. Por ejemplo, `int semAforo=-1`. En la manejadora de SIGINT, sólo si semAforo vale distinto de -1, elimináis el recurso con `semctl`. Esto es lógico: si vale -1 es porque no se ha creado todavía o porque al intentar crearlo la llamada al sistema devolvió error. En ambos casos, no hay que eliminar el recurso.
- VI. Para evitar que todos los identificadores de recursos tengan que ser variables globales para que los vea la manejadora de SIGINT, podéis declarar una estructura que los contenga a todos y así sólo gastáis un identificador del espacio de nombres globales.
- VII. A muchos os da el error "*Interrupted System Call*". Mirad la sesión de "Sucesos Asíncronos", en el apartado "Llamadas al sistema interrumpidas (bloqueantes y no bloqueantes)". Allí se explica lo que pasa con `wait`. A vosotros os pasa con `semop`, pero es lo mismo. Aprovechad la solución que se propone al final del apartado.
- VIII. Al acabar la práctica, con CTRL+C, al ir a borrar los recursos IPC, puede ser que os ponga "*Invalid argument*", pero, sin embargo, se borren bien. La razón de esto es que habéis registrado la manejadora de SIGINT para todos los procesos. Al pulsar CTRL+C, la señal la reciben todos, el padre y los otros procesos. El primero que obtiene la CPU salta a su manejadora y borra los recursos. Cuando saltan los demás, intentan borrarlos, pero como ya están borrados, os da el error. Esto debería evitarse, configurando la señal tras la creación de los hijos o similar.
- IX. Se os recuerda que, si ponéis señales para sincronizar esta práctica, la nota bajará. Usad semáforos, que son mejores para este cometido, y de eso se trata la práctica también.
- X. Todos vosotros, tarde o temprano, os encontraréis con un error que no tiene explicación: un proceso que desaparece, un semáforo que parece no funcionar, etc. La actitud en este caso no es tratar de justificar la imposibilidad del error. Así no lo encontraréis. Tenéis que ser muy sistemáticos. Hay un árbol entero de posibilidades de error y no tenéis que descartar ninguna de antemano, sino ir podando ese árbol. Tenéis que encontrar a los procesos responsables y tratar de localizar la línea donde se produce el error. Si el error es "*Segmentation fault. Core dumped*", es que estáis accediendo a zonas de memoria no permitidas para el proceso, y para localizar el

problema no os quedará más remedio que depurar mediante órdenes de impresión dentro del código.

Para ello, insertad líneas del tipo:

```
fprintf(stderr, "...", ...);
```

donde sospechéis que hay problemas. En esas líneas identificad siempre al proceso que imprime el mensaje. Comprobad todas las hipótesis, hasta las más evidentes. Cuando ejecutéis la práctica, redirigid el canal de errores a un fichero con `2>salida.txt`, por ejemplo.

Si cada proceso pone un identificador de tipo "P1", "P2", etc. en sus mensajes, podéis quedaros con las líneas que contienen esos caracteres con:

```
grep "P1" salida > salida2
```

- XI. Si aparecen errores de acceso a memoria, especialmente a partir de la versión 2 de la biblioteca y en la función *isValidHandle*, puede ser porque creais los procesos auxiliares antes de llamar a `PARKING_inicio`. Todos los procesos se deben crear después de llamar a `PARKING_inicio`.
- XII. Un posible problema de concurrencia que hay que considerar, sucede cuando la biblioteca nos pregunta en qué posición se debe aparcar un coche. Mientras se está realizando la pregunta podría la situación cambiar porque un chófer, que es un proceso distinto, desapareciera justo un coche. Para ver si el problema se produce o no, debéis observar el pseudocódigo de la biblioteca proporcionado más arriba. En cualquier caso, lo que es evidente es que el control de la ocupación de la acera debe de ser acorde con el que realiza la biblioteca. Para simplificar el problema, la biblioteca libera una zona de la acera no cuando el coche pasa, en un primer golpe, a estar sobre las líneas de la calzada ($y=1$), sino cuando pasa efectivamente a ocupar el carril de circulación ($y=2$).
- XIII. En la biblioteca puede ocurrir una terminación anormal (cuando hay un choque, por ejemplo) o una terminación normal (cuando se agota el tiempo). En este segundo caso, lo que debe hacer el proceso hijo encargado de controlar el tiempo antes de morir es llamar a la función de la biblioteca **PARKING_fin(1)**. Esto no significa que la práctica se detenga inmediatamente. Los coches deben seguir desapareciéndose hasta que la acera quede vacía. Por lo tanto, no debemos matar inmediatamente a los chóferes. Cuando hayan acabado su trabajo, el padre volverá de su llamada a **PARKING_simulaciOn()** y podéis eliminar los procesos, esperar por ellos y eliminar los recursos reservados.
- XIV. Respecto a la función **PARKING_fin**, en terminación anormal (cuando se le pasa un cero), la función prácticamente no hace nada. De este modo, se trata de conservar las condiciones que hay en el momento de la finalización. Es, por tanto, responsabilidad de vuestro programa matar los procesos, esperar por ellos, liberar recursos, etc.